

Attendance Capture System — Architecture & Design

Project: DST Design Thinking — NUST 2026

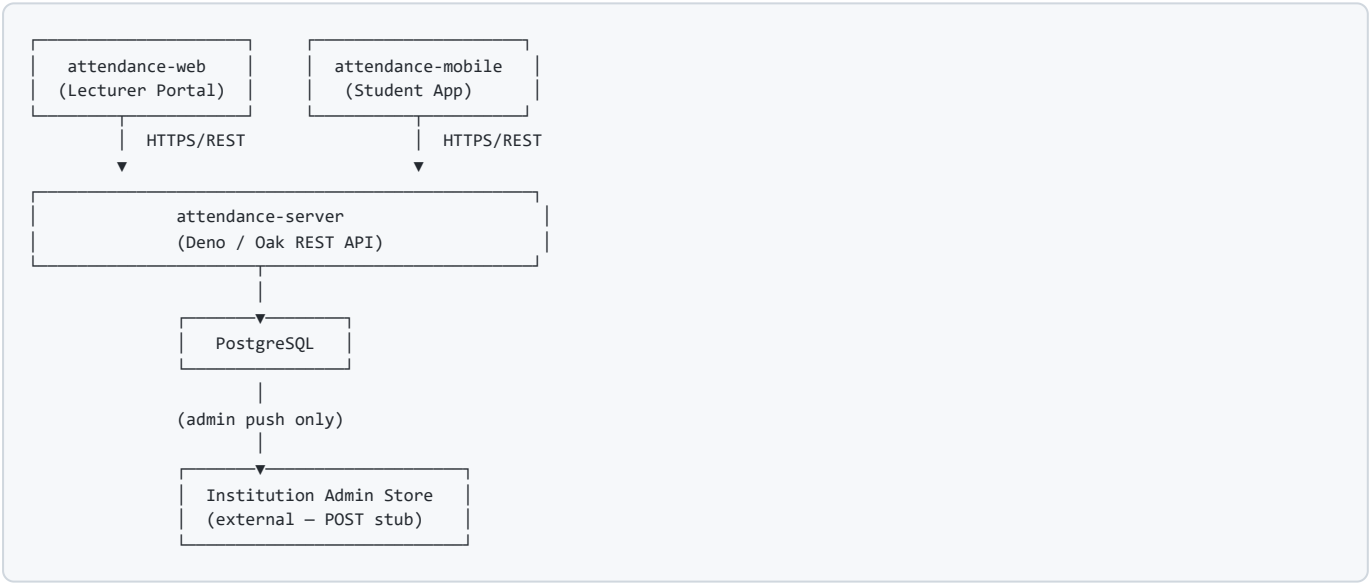
Stack: Deno/Oak/Postgres · React/Vite · React Native

Date: May 2026

1. System Overview

Three separate repos/projects:

Project	Audience	Tech
attendance-server	API backend	Deno, Oak, Postgres
attendance-web	Lecturers	React, Vite, TypeScript
attendance-mobile	Students	React Native, TypeScript



2. User Roles

Role	App	Capabilities
STUDENT	Mobile	Register, login, scan QR, capture attendance, view history
LECTURER	Web	Register, login, manage classes, view attendance, download PDF, push to admin
ADMIN	Web	Full system oversight — manage all users, view all classes and attendance across all lecturers, view push logs, trigger admin pushes, manage other admins

ADMIN accounts are not self-registered. The first admin is created via a seed script in the migration. Subsequent admins can only be created by an existing `ADMIN`.

3. Authentication

- **Mechanism:** JWT (access token + refresh token)
- **Access token TTL:** 15 minutes
- **Refresh token TTL:** 7 days (stored in DB for revocation)
- **Password hashing:** bcrypt (cost factor 12)
- Students register with: `full_name`, `student_number`, `email`, `password` + encrypted device header
- Lecturers register with: `full_name`, `staff_number`, `email`, `password`
- Admins are created by other admins via `POST /admin/users`
- All roles share the same `/auth/login` endpoint — role is encoded in the JWT payload

JWT Payload Shape

```
{
  "sub": "<user_uuid>",
  "role": "STUDENT | LECTURER | ADMIN",
  "iat": 1716000000,
  "exp": 1716000900
}
```

4. Device Binding

4.1 Transport — Encrypted Header

Device info is **never placed in the request body**. It travels as a custom HTTP header `x-Device-Info` that is encrypted at the application layer, in addition to TLS. This makes it opaque to anyone intercepting or inspecting traffic (including TLS-terminating proxies / debugging proxies).

Encryption scheme: AES-256-GCM

Key: 32-byte random key shared between the server and the mobile app build (`DEVICE_ENCRYPTION_KEY` env var, baked into the app at build time)

Header format: `X-Device-Info: <base64(IV || ciphertext || authTag)>`

- IV: 12 random bytes (generated fresh per request)
- ciphertext: AES-256-GCM encrypted UTF-8 JSON
- authTag: 16-byte GCM authentication tag appended after ciphertext

The server's `device.middleware.ts` decodes, decrypts, and attaches the parsed object to `ctx.state.deviceInfo` before the route handler runs. A missing header or failed decryption returns `400 Bad Request`.

4.2 Device Info Payload (plaintext before encryption)

```
{
  "device_fingerprint": "unique-device-id",
  "device_model": "Samsung Galaxy A55",
  "os": "android",
  "os_version": "14",
  "app_version": "1.0.0"
}
```

4.3 Behaviour

- Device info is required on **registration** and **every attendance capture** request
- The server records it in the `devices` table on registration and in `attendance_records` on each capture
- The server records but does **not block** on device mismatch (capture-only policy)
- `last_seen_at` is updated on every authenticated request from the mobile app

5. QR Code Flow

5.1 Lecturer side (web app)

1. Lecturer creates a class (code, subject, venue, scheduled time)
2. Optionally sets a challenge code (short alphanumeric, e.g., `BLUE42`)
3. Server generates a **signed QR token** (HMAC-SHA256) and stores it with the class
4. Web app renders the QR code from the token — lecturer prints it and posts it in the classroom
5. If a challenge code was set, the lecturer **announces it verbally** or writes it on the board

5.2 QR Token payload (encoded into QR image)

```
{
  "class_id": "<uuid>",
  "challenge_required": true,
  "iat": 1716000000
}
```

The payload is **HMAC-SHA256 signed** with a server secret. The QR image itself does not expire — the lecturer is responsible for managing the printed sheet. The server can optionally validate `scheduled_at` drift server-side.

5.3 Student side (mobile app)

1. Student opens scan screen → camera opens
 2. Student scans QR → app decodes the signed token
 3. App sends token to server for verification: server checks HMAC signature and extracts `class_id` and `challenge_required`
 4. If `challenge_required: true` → app prompts student to enter the challenge code
 5. App collects device info + geolocation
 6. App submits attendance payload (see Section 7.2)
-

6. Geolocation

- **Mandatory** — the attendance submission is rejected (`422`) if coordinates are not provided
 - Collected at the moment the student submits attendance
 - Stored as `latitude` / `longitude` (decimal degrees, 8 decimal places)
 - **No server-side proximity validation** — coordinates are recorded only
 - Mobile app requests location permission on first launch; the scan flow is blocked until permission is granted
-

7. Database Schema

7.1 Full SQL

```
-- =====
-- USERS
-- =====
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  full_name   VARCHAR(255) NOT NULL,
  role        VARCHAR(20) NOT NULL CHECK (role IN ('STUDENT', 'LECTURER', 'ADMIN')),
  student_number VARCHAR(50) UNIQUE, -- only populated for STUDENT
  staff_number VARCHAR(50) UNIQUE, -- only populated for LECTURER
  active      BOOLEAN NOT NULL DEFAULT TRUE, -- ADMIN
);

-- =====
-- DEVICES (student device registrations)
-- =====
CREATE TABLE devices (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  device_fingerprint VARCHAR(255) NOT NULL,
  device_model  VARCHAR(255),
  os           VARCHAR(50), -- 'ios' | 'android'
  os_version   VARCHAR(50),
  app_version  VARCHAR(50),
  registered_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_seen_at TIMESTAMPTZ
);

-- =====
-- REFRESH TOKENS (for revocation support)
-- =====
CREATE TABLE refresh_tokens (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  token_hash  VARCHAR(255) NOT NULL UNIQUE,
  issued_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  expires_at  TIMESTAMPTZ NOT NULL,
  revoked     BOOLEAN NOT NULL DEFAULT FALSE
);

-- =====
-- CLASSES
-- =====
CREATE TABLE classes (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  lecturer_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  class_code  VARCHAR(100) NOT NULL,
  subject_name VARCHAR(255) NOT NULL,
  venue       VARCHAR(255),
  scheduled_at TIMESTAMPTZ NOT NULL,
  challenge_code VARCHAR(100), -- NULL = no challenge
  challenge_required BOOLEAN NOT NULL DEFAULT FALSE,
  qr_token     TEXT NOT NULL, -- signed token string encoded in QR image
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- =====
-- ATTENDANCE RECORDS
-- =====
CREATE TABLE attendance_records (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  class_id     UUID NOT NULL REFERENCES classes(id) ON DELETE CASCADE,
  student_id   UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  device_fingerprint VARCHAR(255) NOT NULL,
  device_model  VARCHAR(255),
  os           VARCHAR(50),
  os_version   VARCHAR(50),
  app_version  VARCHAR(50),
  latitude     NUMERIC(10, 8) NOT NULL,
  longitude    NUMERIC(11, 8) NOT NULL,
  captured_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
);
```

```

-- Fraud detection: set when the device fingerprint is already associated
-- with a different student's account or a previous attendance record
flagged          BOOLEAN          NOT NULL DEFAULT FALSE,
flag_reason      VARCHAR(100),    -- e.g. 'DEVICE_CONFLICT'

-- A student may only appear once per class
CONSTRAINT uq_class_student UNIQUE (class_id, student_id)
);

-- =====
-- ADMIN PUSH LOGS
-- =====

CREATE TABLE admin_push_logs (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  class_id      UUID          NOT NULL REFERENCES classes(id) ON DELETE CASCADE,
  pushed_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  status        VARCHAR(20) NOT NULL CHECK (status IN ('SUCCESS', 'FAILED')),
  http_status   INTEGER,
  response_body TEXT
);

-- =====
-- INDEXES
-- =====

CREATE INDEX idx_classes_lecturer      ON classes(lecturer_id);
CREATE INDEX idx_attendance_class      ON attendance_records(class_id);
CREATE INDEX idx_attendance_student    ON attendance_records(student_id);
CREATE INDEX idx_attendance_device     ON attendance_records(device_fingerprint);
CREATE INDEX idx_attendance_flagged    ON attendance_records(class_id, flagged) WHERE flagged = TRUE;
CREATE INDEX idx_devices_user          ON devices(user_id);
CREATE INDEX idx_devices_fingerprint  ON devices(device_fingerprint);
CREATE INDEX idx_refresh_tokens_user   ON refresh_tokens(user_id);

```

7.2 Entity Relationship (summary)

```

users 1——< devices
users (LECTURER) 1——< classes
classes 1——< attendance_records >——1 users (STUDENT)
classes 1——< admin_push_logs
users 1——< refresh_tokens

```

8. API Contract

Base URL: `https://<host>/api/v1`

All protected routes require: `Authorization: Bearer <access_token>`

8.1 Auth

Method	Route	Auth	Description
POST	<code>/auth/register/student</code>	None	Student registration + device binding
POST	<code>/auth/register/lecturer</code>	None	Lecturer registration
POST	<code>/auth/login</code>	None	Login (both roles)
POST	<code>/auth/refresh</code>	None	Refresh access token
POST	<code>/auth/logout</code>	Required	Revoke refresh token

POST /auth/register/student

Headers:

```
X-Device-Info: <base64(IV || ciphertext || authTag)>
```

Request body:

```
{
  "full_name": "Jane Doe",
  "student_number": "221045678",
  "email": "jane@students.nust.na",
  "password": "SecurePass123!"
}
```

- Device info travels encrypted in the `X-Device-Info` header (see Section 4)
- Server decrypts the header, records device in `devices` table

Response `201` :

```
{
  "access_token": "eyJ...",
  "refresh_token": "eyJ..."
}
```

POST /auth/register/lecturer

Request:

```
{
  "full_name": "Prof. Smith",
  "staff_number": "ST00123",
  "email": "smith@nust.na",
  "password": "SecurePass123!"
}
```

Response `201` : same as student registration

POST /auth/login

Request:

```
{
  "email": "jane@students.nust.na",
  "password": "SecurePass123!"
}
```

Response `200` :

```
{
  "access_token": "eyJ...",
  "refresh_token": "eyJ...",
  "user": {
    "id": "uuid",
    "full_name": "Jane Doe",
    "role": "STUDENT"
  }
}
```

POST /auth/refresh

Request:

```
{ "refresh_token": "eyJ..." }
```

Response 200 :

```
{ "access_token": "eyJ..." }
```

8.2 Classes (LECTURER only)

Method	Route	Description
POST	/classes	Create a new class
GET	/classes	List all classes for the authenticated lecturer
GET	/classes/:id	Get class details
PUT	/classes/:id	Update class details
DELETE	/classes/:id	Delete a class
GET	/classes/:id/attendance	List attendance records for the class
GET	/classes/:id/report	Download attendance PDF report
POST	/classes/:id/push-admin	Push attendance log to institution admin store

POST /classes

Request:

```
{
  "class_code": "DST301",
  "subject_name": "Design Thinking",
  "venue": "Block A Room 204",
  "scheduled_at": "2026-06-01T08:00:00Z",
  "challenge_code": "BLUE42"
}
```


- `challenge_code` is optional. If omitted, `challenge_required` will be `false`.

Response 201 :

```
{
  "id": "uuid",
  "class_code": "DST301",
  "subject_name": "Design Thinking",
  "venue": "Block A Room 204",
  "scheduled_at": "2026-06-01T08:00:00Z",
  "challenge_required": true,
  "qr_token": "<signed-token-string>",
  "created_at": "2026-05-24T10:00:00Z"
}
```

The `qr_token` value is what the web app encodes into the printable QR image.

GET /classes/:id/attendance

Supports optional query param `?flagged=true` to return only suspicious records.

Response 200 :

```
{
  "class_id": "uuid",
  "class_code": "DST301",
  "subject_name": "Design Thinking",
  "scheduled_at": "2026-06-01T08:00:00Z",
  "total_attended": 32,
  "flagged_count": 1,
  "records": [
    {
      "student_id": "uuid",
      "full_name": "Jane Doe",
      "student_number": "221045678",
      "captured_at": "2026-06-01T08:07:22Z",
      "latitude": -22.55892300,
      "longitude": 17.07336200,
      "device_model": "Samsung Galaxy A55",
      "os": "android",
      "flagged": true,
      "flag_reason": "DEVICE_CONFLICT"
    }
  ]
}
```

- `flagged_count` is always included in the response so the lecturer sees at a glance how many suspicious records exist without reading through the full list
- The PDF report (Section 8.2 GET /classes/:id/report) visually marks flagged rows (e.g., highlighted or annotated with "⚠ Device Conflict")

GET /classes/:id/report

- Returns a **PDF file** (`Content-Type: application/pdf`)
- Filename: `attendance-DST301-20260601.pdf`
- Content: class details header, table of students (name, student number, captured time, device)

POST /classes/:id/push-admin

- Sends a POST request to the institution admin store endpoint (configured via env var)
- Logs the result in `admin_push_logs`

Response `200` :

```
{
  "status": "SUCCESS",
  "http_status": 200,
  "pushed_at": "2026-06-01T09:00:00Z"
}
```

8.3 Attendance (STUDENT only)

Method	Route	Description
POST	<code>/attendance</code>	Submit attendance for a class
GET	<code>/attendance/history</code>	Get student's attendance history

POST /attendance

Headers:

```
Authorization: Bearer <access_token>
X-Device-Info: <base64(IV || ciphertext || authTag)>
```

Request body:

```
{
  "qr_token": "<signed-token-from-qr-scan>",
  "challenge_code": "BLUE42",
  "latitude": -22.55892300,
  "longitude": 17.07336200
}
```

- `challenge_code` is required only if `challenge_required: true` in the decoded QR token
- `latitude` and `longitude` are **mandatory** — submission is rejected if either is absent
- Device info travels encrypted in the `X-Device-Info` header (see Section 4)

Server validation steps:

1. Verify JWT of the student (auth middleware)
2. Decrypt and parse `X-Device-Info` header (device middleware)
3. Verify HMAC signature of `qr_token`
4. Extract `class_id` from token
5. If `challenge_required`, verify `challenge_code` matches `classes.challenge_code`
6. Verify `latitude` and `longitude` are present
7. Check `UNIQUE(class_id, student_id)` — reject duplicate with `409 Conflict`
8. **Device conflict check** — query both `devices` and `attendance_records` for the incoming `device_fingerprint`:
 - If any row is found where `device_fingerprint` matches but `user_id / student_id` is a **different student**, set `flagged = true`, `flag_reason = 'DEVICE_CONFLICT'`
 - Attendance is still recorded regardless — this is a **non-blocking warning**
9. Insert `attendance_records` row (with `flagged` and `flag_reason` set accordingly)

Response `201` :

```
{
  "id": "uuid",
  "class_id": "uuid",
  "class_code": "DST301",
  "subject_name": "Design Thinking",
  "captured_at": "2026-06-01T08:07:22Z"
}
```

Error responses:

- `400` — missing or invalid `X-Device-Info` header
- `400` — invalid QR token signature
- `400` — wrong challenge code
- `409` — attendance already captured for this class
- `422` — challenge code required but not provided
- `422` — latitude or longitude missing

GET /attendance/history

Response `200` :

```
{
  "records": [
    {
      "id": "uuid",
      "class_code": "DST301",
      "subject_name": "Design Thinking",
      "venue": "Block A Room 204",
      "lecturer_name": "Prof. Smith",
      "scheduled_at": "2026-06-01T08:00:00Z",
      "captured_at": "2026-06-01T08:07:22Z"
    }
  ]
}
```

8.4 Admin Routes (ADMIN only)

Method	Route	Description
GET	/admin/users	List all users (students + lecturers + admins)
GET	/admin/users/:id	Get a single user's profile
PUT	/admin/users/:id	Update a user (e.g., deactivate)
DELETE	/admin/users/:id	Delete a user
POST	/admin/users	Create a new ADMIN account
GET	/admin/classes	List all classes across all lecturers
GET	/admin/classes/:id/attendance	View attendance for any class
GET	/admin/attendance	List all attendance records system-wide
GET	/admin/push-logs	View all admin push logs
POST	/admin/classes/:id/push-admin	Manually trigger admin push for any class

GET /admin/users

Response 200 :

```
{
  "users": [
    {
      "id": "uuid",
      "full_name": "Jane Doe",
      "email": "jane@students.nust.na",
      "role": "STUDENT",
      "student_number": "221045678",
      "staff_number": null,
      "created_at": "2026-05-01T09:00:00Z"
    }
  ]
}
```

POST /admin/users (create admin account)

Request:

```
{
  "full_name": "Admin User",
  "email": "admin@nust.na",
  "password": "SecurePass123!"
}
```

Response 201 : standard user object with role: "ADMIN"

PUT /admin/users/:id

Request (partial update — only send fields to change):

```
{
  "full_name": "Updated Name",
  "active": false
}
```

Requires adding an `active BOOLEAN NOT NULL DEFAULT TRUE` column to `users` — see schema note below.

9. Admin Push Stub

The admin push endpoint is a **configurable POST request** defined in an env var:

```
ADMIN_STORE_URL=https://admin.institution.na/api/attendance
ADMIN_STORE_API_KEY=<key>
```

Payload sent:

```
{
  "class_code": "DST301",
  "subject_name": "Design Thinking",
  "lecturer": {
    "staff_number": "ST00123",
    "full_name": "Prof. Smith"
  },
  "scheduled_at": "2026-06-01T08:00:00Z",
  "records": [
    {
      "student_number": "221045678",
      "full_name": "Jane Doe",
      "captured_at": "2026-06-01T08:07:22Z"
    }
  ]
}
```

This stub can be updated later to match whatever API the institution exposes.

10. Project Directory Structure

10.1 attendance-server

```
attendance-server/
├─ deno.json                # tasks, imports map
├─ deno.lock
├─ .env.example
├─ migrations/
│   └─ 001_initial_schema.sql
├─ src/
│   ├─ app.ts                # Oak app + middleware registration
│   ├─ main.ts               # entry point
│   ├─ db.ts                 # Postgres connection pool
│   ├─ config.ts             # env var loading
│   └─ middleware/
│       ├─ auth.middleware.ts # JWT verification + role guard
│       ├─ device.middleware.ts # decrypt X-Device-Info header → ctx.state.deviceInfo
│       └─ cors.middleware.ts
│   └─ routes/
│       ├─ auth.routes.ts
│       ├─ classes.routes.ts
│       ├─ attendance.routes.ts
│       └─ admin.routes.ts
│   └─ controllers/
│       ├─ auth.controller.ts
│       ├─ classes.controller.ts
│       ├─ attendance.controller.ts
│       └─ admin.controller.ts
│   └─ services/
│       ├─ auth.service.ts
│       ├─ classes.service.ts
│       ├─ attendance.service.ts
│       ├─ admin.service.ts
│       ├─ pdf.service.ts
│       └─ admin-push.service.ts
│   └─ utils/
│       ├─ jwt.ts            # sign / verify access + refresh tokens
│       ├─ qr-token.ts       # HMAC sign / verify QR tokens
│       ├─ device-crypto.ts  # AES-256-GCM encrypt/decrypt for device info
│       └─ hash.ts           # bcrypt helpers
```

10.2 attendance-web

```
attendance-web/
├── package.json
├── vite.config.ts
├── tsconfig.json
├── index.html
├── src/
│   ├── main.tsx
│   ├── App.tsx
│   ├── api/
│   │   ├── client.ts          # axios instance with token refresh interceptor
│   │   ├── auth.api.ts
│   │   ├── classes.api.ts
│   │   └── attendance.api.ts
│   ├── context/
│   │   └── AuthContext.tsx
│   ├── hooks/
│   │   ├── useClasses.ts
│   │   └── useAttendance.ts
│   ├── pages/
│   │   ├── LoginPage.tsx
│   │   ├── RegisterPage.tsx
│   │   ├── DashboardPage.tsx
│   │   ├── ClassesPage.tsx
│   │   ├── CreateClassPage.tsx
│   │   ├── ClassDetailPage.tsx
│   │   └── admin/
│   │       ├── AdminDashboardPage.tsx
│   │       ├── UsersPage.tsx
│   │       ├── AllClassesPage.tsx
│   │       ├── AllAttendancePage.tsx
│   │       └── PushLogsPage.tsx
│   ├── components/
│   │   ├── QRCodeDisplay.tsx # renders printable QR from token
│   │   ├── AttendanceTable.tsx
│   │   └── shared/
│   ├── types/
│   └── index.ts
```

10.3 attendance-mobile

```
attendance-mobile/
├── package.json
├── tsconfig.json
├── app.json          # Expo config
├── App.tsx
├── src/
│   ├── api/
│   │   ├── client.ts          # axios + token refresh
│   │   ├── auth.api.ts
│   │   └── attendance.api.ts
│   ├── context/
│   │   └── AuthContext.tsx
│   ├── navigation/
│   │   └── AppNavigator.tsx
│   ├── screens/
│   │   ├── LoginScreen.tsx
│   │   ├── RegisterScreen.tsx
│   │   ├── HomeScreen.tsx
│   │   ├── ScanScreen.tsx     # camera + QR decode
│   │   ├── ChallengeScreen.tsx
│   │   └── HistoryScreen.tsx
│   ├── hooks/
│   │   └── useAttendanceHistory.ts
│   ├── utils/
│   │   ├── device.ts          # collect device fingerprint + info
│   │   └── geo.ts             # request + read geolocation
│   ├── types/
│   └── index.ts
```

11. Key Libraries

attendance-server (Deno)

Purpose	Library
HTTP framework	<code>oak</code>
Postgres client	<code>postgres</code> (deno-postgres)
JWT	<code>djwt</code> (Deno std/crypto compatible)
Password hashing	<code>bcrypt</code>
HMAC (QR token)	Deno native <code>crypto.subtle</code>
PDF generation	<code>pdf-lib</code>

attendance-web (React/Vite)

Purpose	Library
HTTP client	<code>axios</code>
Routing	<code>react-router-dom</code>
QR code rendering	<code>qrcode.react</code>
Forms	<code>react-hook-form</code>
PDF download	server-generated (streamed from API)

attendance-mobile (React Native / Expo)

Purpose	Library
HTTP client	<code>axios</code>
Navigation	<code>@react-navigation/native</code>
QR scanner	<code>expo-camera</code> + <code>expo-barcode-scanner</code>
Geolocation	<code>expo-location</code>
Device info	<code>expo-device</code> + <code>expo-application</code>
Secure storage	<code>expo-secure-store</code> (tokens)
Device info encryption	<code>expo-crypto</code> (AES-256-GCM via <code>expo-crypto</code> + <code>TextEncoder</code>)

12. Environment Variables (.env.example — server)

```
# Server
PORT=3000

# Postgres
DATABASE_URL=postgres://user:password@localhost:5432/attendance_db

# JWT
JWT_ACCESS_SECRET=change-me-access
JWT_REFRESH_SECRET=change-me-refresh
JWT_ACCESS_TTL_MINUTES=15
JWT_REFRESH_TTL_DAYS=7

# QR Token signing
QR_HMAC_SECRET=change-me-qr

# Device info encryption (AES-256-GCM — 32-byte hex key, shared with mobile app build)
DEVICE_ENCRYPTION_KEY=change-me-32-byte-hex-key

# Admin push
ADMIN_STORE_URL=https://admin.institution.na/api/attendance
ADMIN_STORE_API_KEY=change-me-admin-key

# CORS
CORS_ORIGIN=http://localhost:5173
```

13. Open Decisions / Notes

#	Topic	Decision
1	PDF library	<code>pdf-lib</code> on server — can swap to HTML template + puppeteer if richer layout needed
2	QR token expiry	Token does not expire by default; lecturer controls sheet lifecycle
3	Admin push auth	Bearer token via <code>ADMIN_STORE_API_KEY</code> — update header strategy when real API is known
4	Mobile framework	Expo (managed workflow) — easiest for device/camera/geo APIs
5	Student number uniqueness	Enforced at DB level (<code>UNIQUE</code>) and validated on registration
6	Device info encryption	AES-256-GCM; shared key baked into mobile build; fresh 12-byte IV per request; travels in <code>X-Device-Info</code> header only
7	ADMIN provisioning	First admin seeded via migration; subsequent admins created by authenticated ADMIN via <code>POST /admin/users</code>
8	Geolocation enforcement	Mobile blocks submission if permission denied; server rejects with <code>422</code> if coordinates absent